Roll No: _____ **Date:** _____

Experiment-11
File Allocation

File allocation is the method an operating system uses to store and manage files on a disk. It decides where and how file data blocks are placed on the storage device so they can be efficiently accessed and updated.

Factors affecting file allocation:

1. **Access time** - How quickly data can be read or written from the disk.
2. **Disk space utilization** - How efficiently the storage space is used.
3. **File size** - Large files may need flexible allocation to avoid fragmentation.
4. **Type of access** - Sequential or random access patterns influence the best allocation method.
5. **File growth** - Whether files expand frequently affects allocation flexibility.
6. **Fragmentation** - How scattered the file blocks are on disk, affecting performance.
7. **Reliability** - The ability to recover files in case of system or disk failures.

File Allocation Methods:

1. Contiguous Allocation

- **Description:** In contiguous allocation, each file is stored in a set of consecutive disk blocks. The directory stores the starting block and the length of the file.
- **Advantages:**
  - Very fast for sequential and direct access, since all blocks are next to each other.
  - Simple to implement and manage.
- **Disadvantages:**
  - Causes external fragmentation as files are created and deleted.
  - Difficult to grow a file if the next blocks are already occupied.
  - Finding large enough continuous space can be time-consuming.

2. Linked Allocation

- **Description:** In this method, each file is a linked list of disk blocks. Each block contains a pointer to the next block of the file, and the directory stores only the starting block address.
- **Advantages:**
  - No external fragmentation since any free block can be used.
  - Files can easily grow by adding more blocks.
- **Disadvantages:**
  - Random access is slow because blocks must be read sequentially using pointers.
  - Pointer storage in each block reduces usable space.
  - If a pointer is lost or damaged, the file becomes corrupted.

3. Indexed Allocation

- **Description:** Here, each file has an index block that keeps the addresses of all its data blocks. The directory stores the address of this index block, which is then used to access file data.
- **Advantages:**
  - Supports direct and random access efficiently.
  - No external fragmentation.
  - File size can grow easily by adding entries to the index block.
- **Disadvantages:**
  - Requires extra space for index blocks.
  - Access involves two steps (index block + data block), which can slow down small files.
  - If the index block is damaged, the whole file becomes inaccessible.

Roll No: _____ **Date:** _____

Program 1: Sequential (Contiguous) File Allocation Method

Aim: Write a program to implement Sequential (Contiguous) File Allocation Method.

Algorithm

1. Start
2. Input total number of disk blocks.
3. Initialize all blocks as free (`used[i]=0`) and `FAT[i]=-1`.
4. Input the number of files. For each file:
   (a) Read the file name, starting block, and number of blocks needed.
   (b) Check if all required blocks are free. If not free, ask again for new details.
   (c) Allocate the blocks:
   - Mark each block as used (1).
   - Update FAT so each block points to the next one.
   - The last block points to -1 (NULL).

   (d) Display that the file is allocated successfully.
5. Display the complete File Allocation Table (FAT).
6. Repeat until user types "exit":
   (a) Ask for a file name.
   (b) Search for the file.
   (c) If found, traverse FAT from its starting block and display all allocated blocks.
   (d) If not found, print "File not found."
7. End

Source Code

```c
#include <stdio.h>
#include <string.h>

struct File {
    char name[20];
    int startBlock; // starting block number
    int count;
};

int main() {
    int totalBlocks, numFiles, i, j;

    printf("Enter total number of disk blocks: ");
```

```
    scanf("%d", &totalBlocks);

    int used[100] = {0}; // 0 = free, 1 = allocated
    int FAT[100]; // File Allocation Table

    for (i = 0; i < totalBlocks; i++)
        FAT[i] = -1; // -1 means free/end

    printf("Enter number of files: ");
    scanf("%d", &numFiles);

    struct File files[10];

    // Step 1: Input file details
    for (i = 0; i < numFiles; i++) {
        printf("\nEnter file name: ");
        scanf("%s", files[i].name);
        printf("Enter starting block: ");
        scanf("%d", &files[i].startBlock);
        printf("Enter number of blocks required: ");
        scanf("%d", &files[i].count);

        // Check if all blocks are free
        int canAllocate = 1;
        for (j = 0; j < files[i].count; j++) {
            if (files[i].startBlock + j >= totalBlocks || used[files[i
].startBlock + j] == 1) {
                canAllocate = 0;
                break;
            }
        }

        if (!canAllocate) {
            printf("Blocks not free! Enter details again.\n");
            i--; // retry
            continue;
        }

        // Mark blocks as used and fill FAT
        for (j = 0; j < files[i].count; j++) {
            used[files[i].startBlock + j] = 1;
            if (j == files[i].count - 1)
                FAT[files[i].startBlock + j] = -1; // last block
            else
                FAT[files[i].startBlock + j] = files[i].startBlock + j
+ 1;
        }

        printf("File '%s' allocated successfully\n", files[i].name);
    }

    // Step 2: Display FAT visually
    printf("\n File Allocation Table (FAT) \n");
    printf("Block: Next Block\n");
    for (i = 0; i < totalBlocks; i++) {
        if (FAT[i] != -1)
            printf("%d: %d\n", i, FAT[i]);
```

```
        else
            printf("%d: NULL\n", i);
    }

    // Step 3: Query file details
    char query[20];
    int found;

    printf("\nEnter file name to display its allocation (type 'exit' to quit): ");
    scanf("%s", query);

    while (1) {
        if (strcmp(query, "exit") == 0)
            break;

        found = 0;
        for (i = 0; i < numFiles; i++) {
            if (strcmp(query, files[i].name) == 0) {
                found = 1;
                printf("\nFile Name: %s\n", files[i].name);
                printf("Blocks (traversing FAT): ");

                int b = files[i].startBlock;
                while (b != -1) {
                    printf("%d", b);
                    b = FAT[b];
                    if (b != -1)
                        printf(" -> ");
                }
                printf(" -> NULL\n");
                break;
            }
        }

        if (!found)
            printf("File not found!\n");

        printf("\nEnter file name to display its allocation (type 'exit' to quit): ");
        scanf("%s", query);
    }

    printf("\nProgram ended.\n");
    return 0;
}
```

Output

```
student@cselab:~$ gcc contiguous.c
student@cselab:~$ ./a.out
Enter total number of disk blocks: 20
Enter number of files: 2

Enter file name: A
```

```
Enter starting block: 2
Enter number of blocks required: 3
File 'A' allocated successfully

Enter file name: B
Enter starting block: 6
Enter number of blocks required: 4
File 'B' allocated successfully

 File Allocation Table (FAT)
Block: Next Block
0: NULL
1: NULL
2: 3
3: 4
4: NULL
5: NULL
6: 7
7: 8
8: 9
9: NULL
... (rest of the blocks show NULL)

Enter file name to display its allocation (type 'exit' to quit): A
File Name: A
Blocks (traversing FAT): 2 -> 3 -> 4 -> NULL

Enter file name to display its allocation (type 'exit' to quit): exit

Program ended.
```

Roll No: _____ **Date:** _____

Program 2: Linked List File Allocation Method

Aim: Write a program to implement linked list file allocation method.

Algorithm

1. Start
2. Input total number of disk blocks.
3. Initialize all blocks as free (`used[i]=0`) and `FAT[i]=-1`.
4. Input number of files. For each file:
   (a) Read file name and number of blocks required.
   (b) Enter the block numbers (may be non-contiguous).
   (c) For each entered block:
   - Check if block is free and valid.
   - If valid, mark it as used (1); else, re-enter the block number.

   (d) Set the first block as the file's starting block.
   (e) Update FAT by linking each block to the next one, and mark the last block as -1 (end).
5. Display the complete File Allocation Table (FAT) showing each block and its next link.
6. Repeat until user types "exit":
   (a) Ask for a file name.
   (b) Search for the file in the list.
   (c) If found, traverse FAT starting from its first block and display all linked blocks.
   (d) If not found, print "File not found."
7. End

Source Code

```c
#include <stdio.h>
#include <string.h>

struct File {
    char name[20];
    int startBlock; // starting block of the file
    int count; // number of blocks
};

int main() {
    int totalBlocks, numFiles, i, j;

```

```c
    printf("Enter total number of disk blocks: ");
    scanf("%d", &totalBlocks);

    int used[100] = {0}; // 0 = free, 1 = allocated
    int FAT[100]; // File Allocation Table

    for (i = 0; i < totalBlocks; i++)
        FAT[i] = -1; // -1 indicates free/end

    printf("Enter number of files: ");
    scanf("%d", &numFiles);

    struct File files[10];
    int blocks[10][20]; // store blocks for each file

    // Step 1: Input file details
    for (i = 0; i < numFiles; i++) {
        printf("\nEnter file name: ");
        scanf("%s", files[i].name);
        printf("Enter number of blocks required: ");
        scanf("%d", &files[i].count);
        printf("Enter the block numbers (can be non-contiguous): ");

        for (j = 0; j < files[i].count; j++) {
            scanf("%d", &blocks[i][j]);
            if (blocks[i][j] >= totalBlocks || used[blocks[i][j]] == 1)
    {
                printf("Block %d invalid or already allocated! Try
    again.\n", blocks[i][j]);
                j--;
                continue;
            }
            used[blocks[i][j]] = 1;
        }

        files[i].startBlock = blocks[i][0];

        // Fill FAT
        for (j = 0; j < files[i].count - 1; j++) {
            FAT[blocks[i][j]] = blocks[i][j + 1]; // point to next
    block
        }
        FAT[blocks[i][files[i].count - 1]] = -1; // last block
    }

    // Step 2: Display FAT visually
    printf("\n File Allocation Table (FAT)\n");
    printf("Block#: Next Block\n");
    for (i = 0; i < totalBlocks; i++) {
        if (FAT[i] != -1)
            printf("%d: %d\n", i, FAT[i]);
        else
            printf("%d: NULL\n", i);
    }

    // Step 3: Query file details
    char query[20];
```

```c
    int found;

    printf("\nEnter file name to display its allocation (type 'exit' to quit): ");
    scanf("%s", query);

    while (1) {
        if (strcmp(query, "exit") == 0)
            break;

        found = 0;
        for (i = 0; i < numFiles; i++) {
            if (strcmp(query, files[i].name) == 0) {
                found = 1;
                printf("\nFile Name: %s\n", files[i].name);
                printf("Blocks (traversing FAT): ");

                int b = files[i].startBlock;
                while (b != -1) {
                    printf("%d", b);
                    b = FAT[b];
                    if (b != -1)
                        printf(" -> ");
                }
                printf(" -> NULL\n");
                break;
            }
        }

        if (!found)
            printf("File not found!\n");

        printf("\nEnter file name to display its allocation (type 'exit' to quit): ");
        scanf("%s", query);
    }

    printf("\nProgram ended.\n");
    return 0;
}
```

Output

```
student@cselab:~$ gcc linked.c
student@cselab:~$ ./a.out
Enter total number of disk blocks: 20
Enter number of files: 2

Enter file name: X
Enter number of blocks required: 3
Enter the block numbers (can be non-contiguous): 1 5 9

Enter file name: Y
Enter number of blocks required: 4
```

```
Enter the block numbers (can be non-contiguous): 2 10 12 14

 File Allocation Table (FAT)
Block#: Next Block
0: NULL
1: 5
2: 10
3: NULL
4: NULL
5: 9
6: NULL
7: NULL
8: NULL
9: NULL
10: 12
11: NULL
12: 14
13: NULL
14: NULL
... (rest are NULL)

Enter file name to display its allocation (type 'exit' to quit): Y
File Name: Y
Blocks (traversing FAT): 2 -> 10 -> 12 -> 14 -> NULL

Enter file name to display its allocation (type 'exit' to quit): exit

Program ended.
```

Roll No: _____ **Date:** _____

Program 3: Indexed File Allocation Method

Aim: Write a program to implement Indexed File Allocation Method.

Algorithm

1. Start
2. Input total number of disk blocks.
3. Initialize all disk blocks as free (`used[i]=0`) and `FAT[i]=-1`.
4. Input the number of files. For each file:
   (a) Read the file name.
   (b) Enter the index block number.
   (c) If the index block is already allocated, ask for another. Mark it as used (1).
   (d) Enter the number of data blocks required.
   (e) Read all data block numbers. For each block, check if it is free.
   (f) If free, mark it as used; otherwise, re-enter.
   (g) Link blocks in FAT:
   - The index block points to the first data block.
   - Each data block points to the next data block.
   - The last data block points to -1 (NULL).
5. Display the File Allocation Table (FAT) showing each block and its next linked block.
6. Repeat until user types "exit":
   (a) Ask for a file name.
   (b) Search the file in the list.
   (c) If found:
   - Display its index block.
   - Traverse FAT starting from the index block and show all linked data blocks.
   (d) If not found, display "File not found."
7. End

Source Code

```c
#include <stdio.h>
#include <string.h>

struct File {
    char name[20];
    int indexBlock; // index block number
    int count; // number of data blocks
};

int main() {
    int totalBlocks, numFiles, i, j;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &totalBlocks);

    int used[100] = {0}; // 0 = free, 1 = allocated
    int FAT[100]; // File Allocation Table

    for (i = 0; i < totalBlocks; i++)
        FAT[i] = -1; // -1 indicates free / end

    printf("Enter number of files: ");
    scanf("%d", &numFiles);

    struct File files[10];
    int blocks[10][20]; // store data blocks for each file

    // Step 1: Input file details
    for (i = 0; i < numFiles; i++) {
        printf("\nEnter file name: ");
        scanf("%s", files[i].name);

        printf("Enter index block number: ");
        scanf("%d", &files[i].indexBlock);

        if (used[files[i].indexBlock] == 1) {
            printf("Index block already allocated! Try another.\n");
            i--; // retry
            continue;
        }
        used[files[i].indexBlock] = 1;

        printf("Enter number of data blocks required: ");
        scanf("%d", &files[i].count);

        printf("Enter data block numbers: ");
        for (j = 0; j < files[i].count; j++) {
            scanf("%d", &blocks[i][j]);
            if (used[blocks[i][j]] == 1) {
                printf("Block %d already allocated! Try another.\n",
   blocks[i][j]);
                j--;
                continue;
            }
            used[blocks[i][j]] = 1;
```

```
        }

        // Fill FAT: index block points to first data block
        FAT[files[i].indexBlock] = blocks[i][0];
        for (j = 0; j < files[i].count - 1; j++) {
            FAT[blocks[i][j]] = blocks[i][j + 1];
        }
        FAT[blocks[i][files[i].count - 1]] = -1; // last block
    }

    // Step 2: Display FAT visually
    printf("\n File Allocation Table (FAT)\n");
    printf("Block#: Next Block\n");
    for (i = 0; i < totalBlocks; i++) {
        if (FAT[i] != -1)
            printf("%d: %d\n", i, FAT[i]);
        else
            printf("%d: NULL\n", i);
    }

    // Step 3: Query file details
    char query[20];
    int found;

    printf("\nEnter file name to display its allocation (type 'exit' to
    quit): ");
    scanf("%s", query);

    while (1) {
        if (strcmp(query, "exit") == 0)
            break;

        found = 0;
        for (i = 0; i < numFiles; i++) {
            if (strcmp(query, files[i].name) == 0) {
                found = 1;
                printf("\nFile Name: %s\n", files[i].name);
                printf("Index Block: %d\n", files[i].indexBlock);
                printf("Blocks (traversing FAT): ");

                int b = FAT[files[i].indexBlock]; // start from first
  data block
                while (b != -1) {
                    printf("%d", b);
                    b = FAT[b];
                    if (b != -1)
                        printf(" -> ");
                }
                printf(" -> NULL\n");
                break;
            }
        }

        if (!found)
            printf("File not found!\n");

        printf("\nEnter file name to display its allocation (type 'exit
```

```
    ' to quit): ");
110         scanf("%s", query);
111     }
112
113     printf("\nProgram ended.\n");
114     return 0;
115 }
```

Output

```
student@cselab:~$ gcc indexed.c
student@cselab:~$ ./a.out
Enter total number of disk blocks: 20
Enter number of files: 2

Enter file name: A
Enter index block number: 4
Enter number of data blocks required: 3
Enter data block numbers: 7 8 1

Enter file name: C
Enter Index Block number: 4
Index block already allocated! Try another.

Enter file name: C
Enter Index Block number: 10
Enter number of data blocks required: 2
Enter data block numbers: 11 22
Block 22 invalid or already allocated! Try another.
12

 File Allocation Table (FAT)
Block#: Next Block
1: NULL
...
4: 7
7: 8
8: 1
10: 11
11: 12
12: NULL
...

Enter file name to display its allocation (type 'exit' to quit): C
File Name: C
Index Block: 10
Blocks (traversing FAT): 11 -> 12 -> NULL
```

```
Enter file name to display its allocation (type 'exit' to quit): exit

Program ended.
```